

Object-Oriented Programming (OOP) Interview Revision Guide

1. What is OOP?

OOP is a programming paradigm that organizes software around objects rather than functions. An object contains: • Attributes (data/state) • Methods (behavior) Benefits: • Reusability • Maintainability • Modularity • Scalability

2. Class vs Object

Class: Blueprint/template for creating objects. Object: Instance of a class. Example: class Car: pass my_car = Car() Interview Answer: A class defines structure and behavior, while an object is a real instance created from that class.

3. Four Pillars of OOP

1. Encapsulation 2. Abstraction 3. Inheritance 4. Polymorphism

4. Encapsulation

Bundling data and methods together while restricting direct access. Benefits: • Data protection • Better control • Easier maintenance Interview Question: Why use encapsulation? Answer: To protect object state and expose only controlled interfaces.

5. Abstraction

Hide implementation details and show only essential functionality. Example: You drive a car without knowing engine internals. Benefits: • Reduced complexity • Cleaner code • Better maintainability

6. Inheritance

Allows a class to acquire properties and methods of another class. Parent/Base Class Child/Derived Class Benefits: • Code reuse • Hierarchical relationships Types: • Single • Multiple • Multilevel • Hierarchical • Hybrid

7. Polymorphism

Same interface, different behavior. Types: • Method Overriding (Runtime) • Method Overloading (Compile-time concept) Example: Different classes implement the same method differently.

8. Constructor

Special method automatically called when an object is created. Python: `__init__()` Purpose: • Initialize object state • Set default values

9. Destructor

Called when object is destroyed. Python: `__del__()` Used for cleanup operations.

10. Association, Aggregation, Composition

Association: General relationship between objects. Aggregation: Weak HAS-A relationship. Child can exist independently. Composition: Strong HAS-A relationship. Child cannot exist without parent. Interview favorite question.

11. Method Overloading vs Overriding

Overloading: Same method name, different parameters. Overriding: Child class redefines parent method. Key Interview Difference: Overriding = Runtime polymorphism.

12. Access Modifiers

Public: accessible everywhere Protected (`_var`): convention-based protection Private (`__var`): name mangling Purpose: Control access to data.

13. Abstract Class vs Interface

Abstract Class: Can contain implemented and abstract methods. Interface: Defines behavior contract. Python commonly uses ABC module.

14. SOLID Principles

S - Single Responsibility Principle O - Open/Closed Principle L - Liskov Substitution Principle I - Interface Segregation Principle D - Dependency Inversion Principle Frequently asked in interviews.

15. Most Common Interview Questions

1. What is OOP? 2. Explain the four pillars. 3. Difference between class and object? 4. Encapsulation vs Abstraction? 5. Inheritance advantages? 6. Overloading vs Overriding? 7. Aggregation vs Composition? 8. Abstract class vs Interface? 9. What is polymorphism? 10. Explain SOLID principles.

16. 30-Second Revision Sheet

Class = Blueprint Object = Instance Encapsulation = Data hiding Abstraction = Hide complexity Inheritance = Code reuse Polymorphism = Same interface, different behavior Constructor = Object initialization Destructor = Object cleanup Aggregation = Weak HAS-A Composition = Strong HAS-A Overloading = Same name, different parameters Overriding = Redefine inherited method SOLID = Foundation of good OOP design